

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	M. Mary Sharlin	Reg. No.:	192511330
Section / Batch:	BE CSE1	Date:	28/08/2026

Code of Conduct

I, M. Mary Sharlin (192511330) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: M. Mary Sharlin

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative Implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)

- a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyse memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)

3. Compare Quadratic Probing and Double Hashing in terms of:

- a) Clustering issues
- b) Collision resolution efficiency
- c) Suitability for dense hash tables

4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)

- a) Practical significance
- b) Impact on algorithm selection

5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)

- a) Space complexity (stack usage)
- b) Ease of implementation
- c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge

Signature: _____

Date: _____

Course Coordinator

Signature: _____

Date: _____

Program Director

Signature: _____

Date: _____

1. Linear probing vs separate chaining.

Basis	Linear Probing	separate chaining
Basic idea	When collision occurs, the next available slot is searched sequentially $h(K), h(K)+1, h(K)+2, \dots$	Each hash-table index stores a linked list/bucket containing all keys that hash to that index.
clustering effect	suffers from primary clustering consecutive occupied location from large blocks.	Does not suffer from primary clustering because collided elements are stored in separate chains.
Memory usage	use only the hash table array, so memory overhead is relatively low	Requires extra memory for linked-list nodes/pointers or pointer structures.
Insertion	Finds the next empty position through sequential probing.	Add the element to the chain at the calculated index.
Implementation	Relatively simple for insertion search, but deletion needs special handling	Requires linked-list/bucket management.
Memory overhead	Low	High due to pointers/node structures.

2. Quick sort vs Merge sort

	Quick sort	Merge sort
Basis		
Basic technique	select a pivot and partitions elements into smaller and larger groups.	Divides the list into halves and merges sorted halves.
Average time	$O(n \log n)$	$O(n \log n)$
Worst-case time	$O(n^2)$ with poor pivot selection	$O(n \log n)$
Best case time	$O(n \log n)$	$O(n \log n)$
Extra memory	Usually low for array implementation, but recursive stack is required	For linked lists, merging can be done efficiently with $O(\log n)$ recursion stack and without copying all nodes
Linked list suitability	Less suitable because random access is unavailable	Highly suitable because sequential traversal is enough
Stability	Normally not stable.	stable when implemented appropriately.
Memory allocation	can avoid extra node allocation	can merge by changing links avoiding creation of duplicate nodes.
10 million nodes	poor pivot choices can make performance very poor	predictable and efficient for such a large linked list

Quadratic probing VS Double Hashing

Basics	Quadratic probing	Double hashing
Basic idea	On collision, probes using quadratic increments such as $h(k)+1^2, h(k)+2^2, h(k)+3^2 \dots$	uses a second hash function to calculate the probe step.
Probe formula	$h(k, i) = (h(k) + c_1 i + c_2 i^2) \text{ mod } m$	$h(k, i) = (h_1(k) + i \times h_2(k)) \text{ mod } m$
Clustering	Eliminates primary clustering but can suffer from secondary clustering.	Greatly reduces both primary and secondary clustering.
Probe sequence	Keys with the same initial hash value may follow the same probe sequence.	Different key can have different probe sequences because of the second hash function.
Efficiency	Good at moderate load factors.	usually provides excellent collision resolution.
Hash functions	Requires one main hash function and constants	Requires two well-designed hash functions.
Deletion	Requires tombstones / special markers	Also requires careful de-handling.
Distribution	Better than linear probing but not as random as double hashing.	Gives a more random like probe sequence.

Best case vs Worst case

	Best case	Worst case
Basis		
Meaning	Minimum time required for favorable time.	Maximum time required for an unfavorable input.
Input condition	Input has a structure that allows the algorithm to finish quickly.	Input causes the maximum number of operations.
Practical significance	Shows the most favorable performance.	Shows the performance guarantee/bound under difficult inputs.
Predictability	May not represent normal performance	Important for system where predictable performance is required.
Algorithm selection	Useful when input is often nearly sorted or favorable.	More important when reliability and guaranteed performance are required.
Reliability	Does not guarantee fast performance	Helps determine an upper performance bound.
Impact on large data	Can be very useful when favorable input occurs frequently	Critical because large datasets magnify poor performance.

Quick sort.

Basis	Recursive	Iterative Quick sort
Implementation	Uses function calls to recursively sort partitions	Uses an explicit stack/data structure to store.
Basic approach	partition $\rightarrow$ recursively sort left $\rightarrow$ recursively sort right.	partition $\rightarrow$ push ranges into an explicit stack $\rightarrow$ process ranges iteratively.
Stack overflow risk	Higher because function-call stack is used.	Lower because stack can be explicitly controlled.
Code readability	Usually shorter and easier to understand.	Usually longer and requires explicit range management.
Memory control	Limited by system call stack.	Programmer has more control over stack memory.
Optimization	Can recurse on smaller partition first and process larger partition iteratively.	Can explicitly push only required partitions.
Performance	Function-call overhead is present but usually	Avoids recursive function-call overhead.